

DATA ANALYTICS IN PYTHON

MODULE 2 - PANDAS

Question Bank - MCQs, Debugging, Optimization & Scenario Challenges

165 MCQs

7 Topics

15 Debug Qs

12 Optimization

12 Scenario

SECTION A - MULTIPLE CHOICE QUESTIONS

TOPIC 1 SERIES & DATAFRAME FUNDAMENTALS (Q1–Q18)

- Q1.** What does `pd.Series([10,20,30])` produce as its index by default?
a) ["a","b","c"] - alphabetical b) [0,1,2] - integer index starting at 0 c) [1,2,3] - one-based integer index d) None - no index until explicitly set
- Q2.** What does `pd.Series({"Lahore":9,"Karachi":16,"Islamabad":2})` use as its index?
a) Integer positions 0,1,2 b) The dict values (9,16,2) c) The dict keys ("Lahore","Karachi","Islamabad") d) A default RangeIndex always overrides dict keys
- Q3.** What does `df["Age"]` return vs `df[["Age"]]`?
a) Both return a DataFrame b) `df["Age"]` returns a Series; `df[["Age"]]` returns a single-column DataFrame c) `df["Age"]` is a 2D object; `df[["Age"]]` is 1D d) They are identical - single vs double brackets have no effect
- Q4.** What does `df.shape` return for a DataFrame with 891 rows and 12 columns?
a) 891 b) 12 c) (891, 12) d) [891, 12]
- Q5.** `df2 = df` vs `df2 = df.copy()` - what is the critical difference?
a) No difference - both are independent copies b) `df2 = df` creates an alias pointing at the same object; modifying `df2` modifies `df`. `.copy()` creates a fully independent DataFrame. c) `df2 = df` is a deep copy; `.copy()` is a shallow copy d) `df2 = df` is faster and memory-efficient in all situations
- Q6.** What does `df.dtypes` show?
a) The number of non-null values per column b) The data type of each column (int64, float64, object, etc.) c) The column names only d) The memory used by each column
- Q7.** What does `pd.DataFrame(arr, columns=["A","B","C"])` require of `arr`?
a) `arr` must be a Python list of lists b) `arr` must have exactly 3 rows c) `arr` must have exactly 3 columns to match the 3 column names d) `arr` must be a dict
- Q8.** `df.sort_values(by="Fare", ascending=False)` - does it modify `df` in place?
a) Yes, `df` is sorted permanently b) No - it returns a new sorted DataFrame; `df` is unchanged unless `inplace=True` is passed c) Yes, but only if `ascending=False` is specified d) No, but it automatically resets the index after sorting
- Q9.** After `sorted_df = df.sort_values("Fare")`, what does `sorted_df.index` look like?
a) [0,1,2,3,...] reset starting from 0 b) The original DataFrame index, re-ordered to match the sorted rows c) An alphabetical label index d) A datetime index
- Q10.** What does `df.rename(columns={"old":"new"})` return?
a) None - renames in place b) A new DataFrame with the renamed column(s) c) A Series of old→new name pairs d) Raises `KeyError` if "old" does not exist by default
- Q11.** What does `df.columns` return?
a) A Python list of strings b) An Index object containing the column labels c) A numpy array of column names d) The number of columns (integer)
- Q12.** `df.memory_usage(deep=True)` vs `deep=False` - key difference for object columns?
a) No difference - object columns are always measured the same b) `deep=False` (the default) underreports memory for object columns by not following string pointers. `deep=True` measures the actual bytes consumed by each string. c) `deep=True` only works for numeric columns d) `deep=False` always overestimates object column memory
- Q13.** What does `df.reset_index(drop=True)` do?
a) Deletes all rows b) Replaces the current index with a new 0,1,2,... RangeIndex; `drop=True` prevents the old index from becoming a column c) Renames the index column to "index" d) Sorts the DataFrame by the current index

Q14. `df.T` on a DataFrame with shape (100, 5) produces what shape?

- a) (100, 5) - unchanged
- b) (5, 100) - rows and columns swapped
- c) (500,) - flattened
- d) Error - T only works on square DataFrames

Q15. What does `df.to_numpy()` return vs `df.values`?

- a) They are completely different
- b) They both return a numpy ndarray of the DataFrame's data. `.to_numpy()` is the preferred modern API; `.values` is the older attribute but still widely used.
- c) `to_numpy()` preserves column names; `.values` does not
- d) `to_numpy()` always returns float64; `.values` preserves dtypes

Q16. When creating a DataFrame from a list of dicts, what determines the column order?

- a) Alphabetical order of keys
- b) The order the keys appear in the FIRST dict (Python 3.7+ preserves insertion order)
- c) The order the dicts appear in the list
- d) Column order is random until you call `sort_index(axis=1)`

Q17. What does `df.assign(new_col=df["A"]*2)` do?

- a) Modifies df in place, adding new_col
- b) Returns a NEW DataFrame with new_col added - df is unchanged
- c) Renames column "A" to "new_col"
- d) Raises an error if "new_col" already exists

Q18. What does `pd.concat([s1, s2])` do when s1 and s2 are Series with overlapping index labels?

- a) Raises an error for duplicate labels
- b) Returns a Series with all values from both, preserving potentially duplicate index labels
- c) Automatically de-duplicates the index
- d) Keeps only the values from s1 where labels overlap

TOPIC 2 READING DATA & FIRST-LOOK FUNCTIONS (Q19–Q34)

Q19. What does `df.info()` display that `df.describe()` does NOT?

- a) Mean and standard deviation
- b) Column dtypes, non-null counts per column, and total memory usage
- c) The 25th, 50th, 75th percentiles
- d) Only numeric columns

Q20. What does `df.describe()` include by default?

- a) ALL columns including text
- b) Only numeric columns (int and float)
- c) Only columns with no missing values
- d) Only columns the user specifies

Q21. What does `df.describe()` output for a numeric column?

- a) Min and max only
- b) count, mean, std, min, 25%, 50%, 75%, max - the 5-number summary plus count, mean, std
- c) Median and mode only
- d) count and sum only

Q22. What is the difference between `df.isnull().sum()` and `df.isnull().mean()`?

- a) They are identical
- b) `sum()` gives count of nulls per column; `mean()` gives proportion (fraction) of nulls per column
- c) `sum()` gives row-wise nulls; `mean()` gives column-wise nulls
- d) `mean()` raises `TypeError` - you cannot call `mean()` on a boolean DataFrame

Q23. `pd.read_csv("file.csv", index_col="patient_id")` - what does `index_col` do?

- a) Adds a new column called "patient_id"
- b) Uses the "patient_id" column as the DataFrame's index instead of the default RangeIndex
- c) Sorts rows by "patient_id"
- d) Drops the "patient_id" column from the result

Q24. What does `pd.read_csv(file, sep=";")` handle?

- a) Files with spaces between fields
- b) Files that use semicolons as the delimiter instead of commas
- c) Files with multiple header rows
- d) Files with no header at all

Q25. `df["col"].value_counts()` - what does it return and in what order?

- a) Each unique value and its count, sorted alphabetically
- b) Each unique value and its count, sorted by count DESCENDING
- c) A boolean Series - True where the value appeared more than once
- d) The total count of the column (a scalar)

Q26. `df["col"].nunique()` returns what?

- a) A list of unique values
- b) The number of distinct non-null values in the column
- c) True if all values are unique, False otherwise
- d) The proportion of unique values

Q27. What does `df.isnull().any(axis=1)` produce?

a) True if the ENTIRE DataFrame has any NaN b) A boolean Series - True for each ROW that contains at least one NaN c) A boolean Series - True for each COLUMN that contains at least one NaN d) The count of NaN values per row

Q28. After `df.head(3)`, which rows are displayed?

a) The 3 rows with the smallest index labels b) The first 3 rows in the current order of the DataFrame c) The 3 rows with the most complete data d) Random 3 rows from the DataFrame

Q29. `df.sample(5, random_state=42)` - what does `random_state` do?

a) Limits sampling to the first 42 rows b) Seeds the random number generator for reproducibility - same seed always picks the same 5 rows c) Samples only from rows where index < 42 d) Shuffles the remaining rows after sampling

Q30. What does `df.info()` report as "Non-Null Count" for a column?

a) The number of rows that are null b) The number of rows that are NOT null - equal to `len(df)` minus the null count c) The total row count including nulls d) The dtype only, not null information

Q31. Why does `df.describe(include="all")` show `'top'` and `'freq'` for object columns?

a) These are error codes for non-numeric columns b) `top` is the most frequent value; `freq` is its frequency count - the only sensible stats for categorical/text data c) `top` and `freq` replace mean and std for objects d) These are metadata fields inserted automatically by Pandas

Q32. `df.drop_duplicates()` with no arguments - what does it check?

a) Only the first column b) All columns - keeps first row among any group of completely identical rows c) The index only d) The last column only

Q33. Which `read_csv` parameter would you use to skip the first two header rows of a file?

a) `nrows=2` b) `header=2` - row index 2 becomes the header c) `skiprows=2` - skip the first 2 rows d) `ignore_index=True`

Q34. `df.memory_usage(deep=True).sum()` - what does the result represent?

a) The number of columns b) Total disk size of the CSV file c) Total bytes actually used by the DataFrame in RAM d) The number of rows multiplied by the number of columns

TOPIC 3 .LOC, .ILOC & BOOLEAN FILTERING (Q35–Q54)

Q35. `df.loc[0:3]` selects how many rows (assuming default `RangeIndex`)?

a) 3 rows (0, 1, 2 - stop is exclusive) b) 4 rows (0, 1, 2, 3 - stop is INCLUSIVE for `loc`) c) 1 row (only row labelled 0) d) Depends on whether the index is sorted

Q36. `df.iloc[0:3]` selects how many rows?

a) 4 rows (0,1,2,3 - `iloc` matches `loc` behavior) b) 3 rows (0, 1, 2 - stop is EXCLUSIVE, just like Python list slicing) c) 1 row (position 0 only) d) 0 rows - `iloc` cannot slice

Q37. After filtering with `filtered = df[df["Fare"] > 100]`, what does `filtered.iloc[0]` give you?

a) The row with index label 0 from the original df b) The FIRST row in filtered (by position), which may have any index label c) An error - `iloc` doesn't work after filtering d) The row with the lowest Fare value

Q38. Why do you need `&` instead of Python's `and` for combining Pandas conditions?

a) `&` is faster than `and` for large DataFrames b) Python's `and/or` expect a single boolean; applied to a Series with multiple elements they raise "the truth value of an array is ambiguous" - `&` works element-wise c) `&` and `and` are identical - this is a style preference only d) `and` works, but returns NaN for missing values

Q39. Why are parentheses required in `df[(df["Age"] > 30) & (df["Pclass"] == 1)]`?

a) Pandas requires all filters to be wrapped in parentheses for any filter b) Operator precedence: `&` binds more tightly than `>` and `==`, so without parentheses the comparison would fail c) Parentheses are optional - they are just a style convention d) The parentheses swap `&` for `and` in this context

Q40. `df["Embarked"].isin(["C","Q"])` checks what?

ANSWER KEY

Q#	Ans	Tag	Explanation
1	B		pd.Series() assigns a RangeIndex (0, 1, 2, ...) when no index is specified.
2	C		When creating a Series from a dict, the keys become the index labels.
3	B		Single bracket on a column name returns a Series (1D). Double brackets return a DataFrame (2D), even for one column.
4	C		.shape returns a tuple (rows, columns) - always two integers.
5	B		.copy() is the safe pattern before any modification - without it, changes to df2 propagate back to df.
6	B		df.dtypes returns a Series where the index is column names and values are dtype objects.
7	C		When specifying column names, the number of names must match the number of columns in arr.
8	B		sort_values() returns a new object by default. Use inplace=True with caution - it is generally discouraged in data pipelines.
9	B		sort_values() preserves original index labels - the integer labels move with their rows. Use .reset_index(drop=True) to get a fresh 0,1,2... index.
10	B		rename() returns a new object by default. Pass inplace=True to modify the original.
11	B		.columns returns a pandas Index object. Use .columns.tolist() to get a plain Python list.
12	B		Verified: object/string columns look tiny with deep=False because Pandas only counts the pointer size, not the pointed-to string data.
13	B		Common after filtering or sorting when you want clean 0-based row labels.
14	B		Transpose swaps rows and columns, just as with NumPy arrays.
15	B		Both return the same underlying array. Use to_numpy() in new code - it has explicit dtype and copy parameters.
16	B		Python 3.7+ guarantees dict insertion order, so columns appear in the order the first dict defines its keys.
17	B		.assign() is the functional, chainable way to add derived columns without mutating the original DataFrame.
18	B		concat() does NOT deduplicate by default. Use ignore_index=True to get a fresh 0,1,2,... index.
19	B		info() is a schema overview - types, nulls, memory. describe() is a statistical summary of values.
20	B		Use describe(include="all") to include object/category columns.
21	B		These are the 8 statistics: count (non-null), mean, std, min, Q1, median, Q3, max.
22	B		Both are idiomatic. mean() * 100 gives the percentage missing per column - a common cleaning audit step.
23	B		Setting index_col is common when loading data with a meaningful identifier column that should not appear as a regular column.
24	B		CSV files exported from European Excel versions often use ";" as separator. Always check the delimiter before loading.
25	B		value_counts() is the fastest way to audit a categorical column. Use dropna=False to include NaN in the count.
26	B		nunique() is frequently used after groupby or to assess cardinality before deciding how to encode a categorical column.
27	B		axis=1 collapses across columns (across the row) - giving one True/False per row. Use this to filter rows that need cleaning.
28	B		head(N) shows the first N rows in the current row order - critical: after sorting, head() reflects the sorted order.

29	B	random_state mirrors numpy's random.seed() - essential for reproducible data splits and sampling.
30	B	info() tells you how many valid (non-null) values each column has - the quickest null audit before deeper cleaning.
31	B	For non-numeric columns, describe() computes count, unique, top (mode), freq (mode frequency) instead of numeric stats.
32	B	Without subset=, it checks every column for exact row duplication - strict deduplication.
33	B	header=N treats row N as the header; skiprows=[0,1] or skiprows=2 skips the first N rows before reading.
34	C	memory_usage(deep=True) follows object/string pointers to measure true RAM consumption - important before loading large datasets.
35	B	Verified: .loc slicing is inclusive on BOTH ends - loc[0:3] gives rows 0, 1, 2, 3.
36	B	Verified: .iloc slicing is exclusive on the stop - matching Python's standard list/range behavior.
37	B	After filtering, .iloc always refers to POSITION within the filtered result. .loc refers to the preserved original labels.
38	B	Verified in class demo: `(df["Age"] > 30) and (df["Pclass"] == 1)` raises ValueError. Use & and for element-wise boolean operations.
39	B	Without parentheses, Python evaluates `30) & (df["Pclass"]` first - causing a TypeError due to incorrect operator grouping.
40	B	isin() is the vectorized equivalent of writing (col=="C") (col=="Q") - cleaner and faster for multi-value membership tests.
41	B	Verified: between() is inclusive on both boundaries by default. Use the inclusive parameter to change this.
42	B	Without na=False, NaN values in the column return NaN (not True/False), which can cause downstream dtype or boolean errors.
43	B	query() takes a string and uses its own expression parser - inside that string, "and"/"or"/"not" are the correct keywords.
44	B	@ is the query() escape for Python variables - it allows external variables to be used inside the query expression string.
45	B	.loc always uses labels, not positions. Both the row and column selectors use label lists here.
46	B	.iloc uses integer positions exclusively. :5 = positions 0,1,2,3,4 and :4 = positions 0,1,2,3.
47	B	drop() returns a new object by default. Pass inplace=True to modify in place (not recommended in modern Pandas pipelines).
48	A	get_dummies() creates N binary columns for N unique values. The drop_first=True parameter drops one to avoid the dummy variable trap for ML.
49	B	np.where with string outputs creates an object-dtype column. Use pd.Categorical() to make it memory-efficient if cardinality is low.
50	B	This is the classic post-filter confusion. Always think: .loc = label, .iloc = position. Labels are preserved after filtering; positions restart at 0.
51	B	`~mask` is Pandas' element-wise NOT. `df[~df["Age"].isnull()]` selects all rows where Age is NOT null.
52	B	Both are functionally identical for row filtering. Use .loc when you also want to select specific columns at the same time.
53	B	Default how="any" removes any row with at least one NaN. Use how="all" to only drop rows that are entirely empty.
54	B	Verified: thresh specifies the minimum number of non-null values required to KEEP a row. A gentler alternative to dropna().
55	B	df.mean() returns a Series of means for numeric columns only. Object columns without a matching mean entry are left as-is.
56	B	errors="coerce" is the production-safe default for type conversion. The pipeline survives; bad values become NaN which you can report.